BCSE307P – Compiler Design Lab

Report Submission

Project Title:

ExpressFlow: Semantic Static Analysis of Express.js Middleware Architectures

Team Member(s):

S. No	Register No	Name	Contributions
1	24BCE0732	Krishna	Planned Contribution: Will implement the lexical and syntactic analysis phase to parse JavaScript source code into Abstract Syntax Trees (AST) and extract route definitions.
2	24BCE2584	Aaditya Jaiswal	Planned Contribution: Will develop the middle-end analysis engine to construct Control-Flow Graphs (CFG), resolve symbols, and verify security invariants.

Problem Identification & Objectives

Problem Statement

In Express.js applications, security checks (such as authentication and authorization) are executed as middleware functions before reaching the target route handler. Because JavaScript attaches middleware dynamically, developers frequently omit required authentication functions on sensitive routes (e.g., leaving `/api/admin` unprotected). This code is syntactically valid and executes without throwing errors, silently bypassing security. Standard linters only check surface syntax, while dynamic security testing requires a running server and active exploits. ExpressFlow addresses this need by statically parsing backend source code, generating a Control-Flow Graph of middleware execution, and detecting omitted security checks ahead of time during the compile/build step.

Objectives

- Parse Express.js source code into an Abstract Syntax Tree (AST) to identify route registrations (`app.get`, `app.post`, `app.use`).
- Construct an Intermediate Representation (IR) structured as a Control-Flow Graph (CFG) representing the sequential execution order of middleware on each route.
- Resolve imported and aliased middleware identifiers using a scoped symbol table.

- Detect and report routes missing mandatory security middleware using path-reachability analysis, outputting line numbers and remediation messages.

Literature Survey

The following 15 relevant papers, tools, and existing systems are summarized below.

S. No.	Title / Author / Year	Methodology	Findings	Gap Identified
1	Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.js Packages — Brito, Ferreira, Monteiro, Lopes, Barros, Fragoso Santos, Santos, 2023	Evaluates static analysis tools for detecting vulnerabilities in Node.js packages using a curated dataset.	Characterized 957 vulnerabilities; tested 9 tools; best three combined detect only 57.6% at 0.11 precision.	Does not analyze Express.js middleware ordering or execution before protected routes.
2	Mining Node.js Vulnerabilities via Object Dependence Graph and Query — Li, Kang, Hou, Cao, 2022	Combines AST, CFG and Object Dependence Graphs (ODG) via abstract interpretation, queried for vulnerability patterns.	ODGEN modeled 13/16 vulnerability types; reported 180 zero-day vulnerabilities; received 70 CVEs.	Focuses on object/data dependencies, not middleware execution order.
3	Static Analysis of Event-Driven Node.js JavaScript Applications — Madsen, Tip, Lhoták, 2015	Constructs event-based call graphs to model callbacks and asynchronous behavior.	Detects bugs from StackOverflow reports with manageable false-positive rates.	No domain-specific analysis of Express.js routes/security middleware chains.
4	Automated Analysis of Security-Critical JavaScript APIs — Taly, Erlingsson, Mitchell, Miller, Nagra, 2011	Static verification tool for reference-monitor API confinement.	Found a previously unknown vulnerability in Yahoo! ADsafe.	Does not model Express.js middleware execution or route-specific requirements.
5	GATEKEEPER: Mostly Static Enforcement of Security and Reliability Policies for JavaScript Code — Guarnieri, Livshits, 2009	Mostly-static policy checker using a Datalog policy language.	1,341 verified warnings across 684 widgets from 9 policies, no false negatives.	Provides general policy checking, not ordered middleware execution paths.
6	Type Analysis for JavaScript — Jensen, Møller, Thiemann, SAS 2009	Abstract interpretation to infer sound type information for full ECMAScript.	Derives static types from dynamically typed JS programs.	Type inference alone cannot verify middleware reachability before handlers.
7	JSAI: A Static Analysis Platform for JavaScript — Kashyap, Dewey, Kuefner, Wagner, Gibbons, Sarracino, Wiedermann, Hardekopf, 2014	Formally specified abstract interpreter combining type, pointer, CFG, string, constant analyses.	General-purpose, configurable platform for JS static analysis.	Lacks Express-specific route/middleware security invariants.
8	Practical Static Analysis of JavaScript Applications in the	Static analysis techniques accounting	Frameworks/libraries hinder precise analysis;	Does not resolve Express.js middleware

	Presence of Frameworks and Libraries — Madsen, Livshits, Fanning, 2013	for external frameworks/libraries.	proposes mitigating techniques.	aliases or ordering.
9	Modeling the HTML DOM and Browser API in Static Analysis of JavaScript Web Applications — Jensen, Madsen, Møller, 2011	Models DOM/browser API behavior for precision in client-side JS analysis.	Improves precision for detecting missing-property and dead-code errors.	Targets browser-side JS, not server-side Express.js middleware.
10	SYNODE: Understanding and Automatically Preventing Injection Attacks on Node.js — Staicu, Pradel, Livshits, NDSS 2018	Static template extraction plus runtime grammar-based policy enforcement.	Studied 235,850 Node.js modules; injection vulnerabilities widespread via eval/exec.	Focuses on injection attacks, not middleware reachability verification.
11	Detecting Node.js Prototype Pollution Vulnerabilities via Object Lookup Analysis — Li, Kang, Hou, Cao, 2021	Flow/context/branch-sensitive static taint analysis (ObjLupAnsys) via object lookup.	Found 61 zero-day vulnerabilities (vs. 18 dynamic, 3 static baseline); 11 CVEs.	Addresses a specific vulnerability class, not general middleware security.
12	NodeMedic: End-to-End Analysis of Node.js Vulnerabilities with Provenance Graphs — Cassel, Wong, Jia, 2023	Provenance graphs tracing interactions and data flows for vulnerability detection and exploit confirmation.	End-to-end pipeline for detecting and confirming Node.js ACE/ACI vulnerabilities.	Focuses on provenance/data flow, not temporal ordering of middleware.
13	NodeMedic-FINE: Automatic Detection and Exploit Synthesis for Node.js Vulnerabilities — Cassel, Sabino, Hsu, Martins, Jia, 2025	Type/structure-aware fuzzer combined with dynamic taint analysis and exploit synthesis.	Evaluated on 33,011 packages; found 2,257 potential flows; synthesized exploits in 766 packages.	Vulnerability-flow analysis, not a lightweight static checker for missing Express.js middleware.
14	ANDROMEDA: Accurate and Scalable Security Analysis of Web Applications — Tripp, Pistoia, Cousot, Cousot, Guarnieri, 2013	Demand-driven static taint analysis via abstract interpretation for Java/.NET/JavaScript.	Scalable, accurate taint analysis in a commercial production tool.	Follows data-flow/taint relationships, not ordered Express.js middleware chains.
15	Bullseye: Detecting Prototype Pollution in NPM Packages with Proof of Concept Exploits — Houis, Jiang, Mannan, Youssef, 2026	Fully automated dynamic analysis using dual side-effect oracles for prototype pollution detection.	Analyzed ~50,000 packages; found zero-day prototype pollution in 290 packages, no false positives.	Targets prototype pollution specifically, not route-level middleware ordering.

Research Gap and Project Motivation

Existing research provides strong techniques for JavaScript and Node.js static analysis, including abstract interpretation, control-flow analysis, object dependence graphs, event-based call graphs, taint analysis, and security-policy checking. However, the reviewed works

generally focus on general JavaScript semantics, specific vulnerability classes, browser security, or dynamic vulnerability discovery. They do not specifically model Express.js route registration and middleware chains as security policies that must be satisfied along every reachable route path. ExpressFlow builds on these program-analysis ideas by extracting Express route and middleware declarations from the AST, resolving middleware symbols, constructing a route-level Control-Flow Graph, and checking whether required security middleware is present on paths leading to sensitive handlers.

Novelty

- Temporal Execution vs. Data Taint: Unlike tools that trace tainted data through variables, ExpressFlow verifies execution order—ensuring security middleware always executes prior to target route handlers.
- Domain-Aware Route CFGs: Instead of generating low-level assembly basic blocks, ExpressFlow builds high-level Control-Flow Graphs where nodes represent Express middleware functions and edges represent HTTP request progression.
- Ahead-of-Time Static Verification: Operates directly on source code during compilation/build time, catching route authorization omissions without running the server or executing test suites.
- Symbol-Aware Middleware Tracking: Resolves imported middleware identifiers across module boundaries (e.g., `const auth = require('./auth')`), preventing false negatives common in pattern-matching linters.